

ASSIGNMENT-6.3

2303a51142

Batch-17

Task Description #1: Classes (Student Class)

Scenario

You are developing a simple student information management module.

Task

- Use an AI tool (GitHub Copilot / Cursor AI / Gemini) to complete a Student class.
- The class should include attributes such as name, roll number, and branch.
- Add a method `display_details()` to print student information.
- Execute the code and verify the output.
- Analyze the code generated by the AI tool for correctness and clarity.

Expected Output #1

- A Python class with a constructor (`__init__`) and a `display_details()` method.
- Sample object creation and output displayed on the console.
- Brief analysis of AI-generated code

```
class Student:
    def __init__(self, name, roll_number, branch):
        self.name = name
        self.roll_number = roll_number
        self.branch = branch

    def display_details(self):
        print("Student Details:")
        print(f"Name       : {self.name}")
        print(f"Roll No    : {self.roll_number}")
        print(f"Branch     : {self.branch}")

# Sample object creation
student1 = Student("Ravi Kumar", "21CSE045", "Computer Science Engineering")
student2 = Student("Sai", "21CSE046", "Civil Engineering")
student3 = Student("Anita", "21CSE047", "Computer Science Engineering")
student4 = Student("Vikas", "21CSE048", "Electronics Engineering")

# Display student information
```

```
student1.display_details()  
student2.display_details()  
student3.display_details()  
student4.display_details()  
print("\n")
```

#OUTPUT

Student Details:

Name : Ravi Kumar

Roll No : 21CSE045

Branch : Computer Science Engineering

Student Details:

Name : Sai

Roll No : 21CSE046

Branch : Civil Engineering

Student Details:

Name : Anita

Roll No : 21CSE047

Branch : Computer Science Engineering

Student Details:

Name : Vikas

Roll No : 21CSE048

Branch : Electronics Engineering

MY ANALYSIS:

Demonstrates object-oriented programming using a class and objects. The constructor initializes student data, and a method is used to display details, improving code reusability and clarity.

Task Description #2: Loops (Multiples of a Number)

Scenario

You are writing a utility function to display multiples of a given number.

Task

- Prompt the AI tool to generate a function that prints the first 10 multiples of a given number using a loop.

- Analyze the generated loop logic.
- Ask the AI to generate the same functionality using another controlled looping structure (e.g., while instead of for).

Expected Output #2

- Correct loop-based Python implementation.
- Output showing the first 10 multiples of a number.
- Comparison and analysis of different looping approaches.

```
def print_multiples(number, limit):
    print(f"Multiples of {number} up to {limit}:")
    for i in range(1, limit + 1):
        multiple = number * i
        print(multiple, end=' ')
    print()
# Sample function call
print_multiples(7, 10)
print_multiples(2, 10)
print_multiples(5, 10)
print_multiples(2, 10)
```

#OUTPUT:

Multiples of 7 up to 10:

7 14 21 28 35 42 49 56 63 70

Multiples of 2 up to 10:

2 4 6 8 10 12 14 16 18 20

Multiples of 5 up to 10:

5 10 15 20 25 30 35 40 45 50

Multiples of 2 up to 10:

2 4 6 8 10 12 14 16 18 20

MY ANALYSIS:

Uses a `for` loop to generate multiples of a given number. The loop runs for a fixed range, making it efficient and easy to understand.

Task Description #3: Conditional Statements (Age Classification)

Scenario

You are building a basic classification system based on age.

Task

- Ask the AI tool to generate nested if-elif-else conditional statements to classify age groups (e.g., child, teenager, adult, senior).
- Analyze the generated conditions and logic.
- Ask the AI to generate the same classification using alternative conditional structures (e.g., simplified conditions or dictionary-based logic).

Expected Output #3

- A Python function that classifies age into appropriate groups.
- Clear and correct conditional logic.
- Explanation of how the conditions work.

```
def classify_age(age):  
    if age < 0:  
        return "Invalid age"  
    elif age <= 12:  
        return "Child"  
    elif age <= 19:  
        return "Teenager"  
    elif age <= 59:  
        return "Adult"  
    else:  
        return "Senior Citizen"  
# Sample function calls  
ages = [10, 15, 25, 65, -5]  
for age in ages:  
    classification = classify_age(age)  
    print(f"Age: {age}, Classification: {classification}")
```

#OUTPUT:

Age: 10, Classification: Child

Age: 15, Classification: Teenager

Age: 25, Classification: Adult

Age: 65, Classification: Senior Citizen

Age: -5, Classification: Invalid age

MY ANALYSIS:

Implements conditional statements to classify age into categories. Proper input validation is included to handle invalid values.

Task Description #4: For and While Loops (Sum of First n Numbers)

Scenario

You need to calculate the sum of the first n natural numbers.

Task

- Use AI assistance to generate a `sum_to_n()` function using a for loop.
- Analyze the generated code.
- Ask the AI to suggest an alternative implementation using a while loop or a mathematical formula.

Expected Output #4

- Python function to compute the sum of first n numbers.
- Correct output for sample inputs.
- Explanation and comparison of different approaches.

```
def sum_of_n_numbers(n):  
    # Using for loop  
    sum_for = 0  
    for i in range(1, n + 1):  
        sum_for += i  
  
    # Using while loop  
    sum_while = 0  
    count = 1  
    while count <= n:  
        sum_while += count  
        count += 1  
  
    return sum_for, sum_while  
# Sample function call  
n = 10  
sum_for, sum_while = sum_of_n_numbers(n)  
print(f"Sum of first {n} natural numbers using for loop: {sum_for}")  
print(f"Sum of first {n} natural numbers using while loop: {sum_while}")
```

#OUTPUT:

Sum of first 10 natural numbers using for loop: 55

Sum of first 10 natural numbers using while loop: 55

MY ANALYSIS

Shows how both `for` and `while` loops can be used to solve the same problem. Helps in understanding loop control and iteration logic.

Task Description #5: Classes (Bank Account Class)

Scenario

You are designing a basic banking application.

Task

- Use AI tools to generate a Bank Account class with methods such as deposit(), withdraw(), and check_balance().
- Analyze the AI-generated class structure and logic.
- Add meaningful comments and explain the working of the code.

Expected Output #5

- Complete Python Bank Account class.
- Demonstration of deposit and withdrawal operations with updated balance.
- Well-commented code with a clear explanation

```
class BankAccount:
    def __init__(self, account_holder, balance=0):
        self.account_holder = account_holder
        self.balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"Deposited: {amount}. New Balance: {self.balance}")
        else:
            print("Deposit amount must be positive.")

    def withdraw(self, amount):
        if 0 < amount <= self.balance:
            self.balance -= amount
            print(f"Withdrew: {amount}. New Balance: {self.balance}")
        else:
            print("Insufficient balance or invalid withdrawal amount.")

    def display_balance(self):
        print(f"Account Holder: {self.account_holder}, Balance: {self.balance}")

# Sample object creation and operations
account = BankAccount("Ravi Kumar", 1000)
account.display_balance()
account.deposit(500)
account.withdraw(300)
account.withdraw(1500)
account.display_balance()
```

#OUTPUT

Account Holder: Ravi Kumar, Balance: 1000

Deposited: 500. New Balance: 1500

Withdrew: 300. New Balance: 1200

Withdraw: 1500

Insufficient balance or invalid withdrawal amount.

MY ANALYSIS

Applies OOP concepts to model basic banking operations. Includes validation to ensure safe deposit and withdrawal operations.